

EAFIX Economic Calendar — Deterministic Execution Rules

Document ID: EAFIX-DES-CAL-001

Version: 1.0

Framework: Deterministic Execution System (DES)

Purpose: Pre-compile all architectural decisions, eliminate runtime ambiguity, and enforce dependency order for the full EAFIX Economic Calendar system.

Each rule targets a specific complexity dependency cluster identified in the system analysis. The cluster reference is noted on every rule.

Table of Contents

1. Pre-Compiled Decisions (DEC-CAL-001 through DEC-CAL-010)
 2. Anti-Pattern Guards
 3. Single-Writer Authority Registry
 4. Phase 0 Deliverable — Database Schema
 5. Build Phase Contracts (P00–P10)
 6. Verification Contracts by Layer
 7. What These Rules Cannot Resolve
-

1. Pre-Compiled Decisions

All ambiguous architectural choices are resolved here exactly once. No downstream phase may re-derive any decision listed in this section. If execution encounters a situation not covered, execution **PAUSES** and a new DEC-CAL-NNN entry is created before proceeding.

decision_id: DEC-CAL-001

```
cluster: "CAL8 identity is the keystone"
category: behavioral
context: "When is CAL8 assigned, and what format does it use?"
options_evaluated:
  - at_download_time: rejected (raw data not yet normalized; currency not finaliz
  - at_db_write_time: rejected (requires identity before upsert can use it as con
  - at_normalization_time: selected
selected_option: at_normalization_time
format: "CAL8-{YYYYMMDD}-{HHMM}-{CCC}-{NNN}"
  # YYYYMMDD: UTC event date
  # HHMM:      UTC event time (24h)
  # CCC:       normalized 3-letter currency code
  # NNN:       collision suffix 001-999 (incremented when slot is taken)
collision_rule: >
  Before assigning NNN=001, query SQLite:
  SELECT COUNT(*) FROM calendar_events
  WHERE cal8 LIKE 'CAL8-{YYYYMMDD}-{HHMM}-{CCC}-%'
  Append next available suffix. Alert if NNN > 010 (unusual clustering).
rationale: >
  CAL8 requires a normalized UTC datetime and currency code, both available
  post-normalization. Assigning here means every downstream component
  (state machine, debounce, EA bridge) has a stable durable key.
encoded_in: services/calendar-ingestor/src/normalizer.py
```

```
decision_id: DEC-CAL-002
cluster: "CHF exclusion creates silent coverage gap"
category: behavioral
context: >
  CHF events are excluded from CAL5/RCI strategy ID generation.
  Do they still receive CAL8 identity and state machine processing?
options_evaluated:
  - drop_chf_at_filter: rejected (EA would trade through CHF events with no signa
  - extend_cal5_to_chf: rejected (breaks RCI regional model; CHF has no major pai
  - cal8_and_state_no_cal5: selected
selected_option: cal8_and_state_no_cal5
rules:
  - CHF events receive CAL8 identity at normalization (same as all events)
  - CHF events pass through state machine (same lifecycle: SCHEDULED→EXPIRED)
  - CHF events are assigned cal5 = "00000" (the universal non-calendar sentinel)
  - CHF events are assigned signal_type = "ECO_UNCAT"
  - CHF events appear in active_calendar_signals.csv with priority_weight calcula
  - EA behavior on ECO_UNCAT: block new trades (same as any active signal)
  - EA behavior on ECO_UNCAT: no matrix parameter lookup attempted
rationale: >
```

CHF events (SNB rate decisions, Swiss CPI) cause meaningful volatility on EURCHF, USDCHF, XAUUSD. Silently dropping them creates an invisible risk.

Assigning ECO_UNCAT preserves blocking behavior without requiring RCI extension
encoded_in: services/calendar-ingestor/src/normalizer.py
 services/calendar-processor/src/cal5_generator.py

decision_id: DEC-CAL-003

cluster: "Two-scheduler SQLite write contention"

category: structural

context: >

calendar-downloader and calendar-processor both need database access.

Concurrent SQLite writers cause SQLITE_BUSY errors and potential corruption.

options_evaluated:

- both_services_write_directly: rejected (write contention; no locking protocol)
- message_queue_between_services: rejected (adds infrastructure complexity before)
- downloader_writes_raw_csv_only: selected

selected_option: downloader_writes_raw_csv_only

rules:

- calendar-downloader writes ONLY to:
 - data/raw/{timestamp}_ff_calendar.csv (raw downloaded content)
 - data/download_state.json (retry/hash state)
- calendar-downloader NEVER opens a SQLite connection
- calendar-processor owns ALL SQLite writes exclusively
- calendar-processor watches data/raw/ via file mtime polling (30s interval)
- calendar-processor ingests new raw CSV files and performs all DB writes
- Single-writer authority for calendar_registry.db: calendar-processor

enforcement: >

Anti-pattern guard CAL-GUARD-002 rejects any SQLite write attempt
from calendar-downloader process at runtime.

rationale: >

Single-writer authority (DES Concept 8) eliminates contention entirely.

No locking protocol needed. Downloader is now stateless except for two files.

encoded_in: services/calendar-downloader/src/storage/file_manager.py
 services/calendar-processor/src/watcher.py

decision_id: DEC-CAL-004

cluster: "Proximity output staleness vs. refresh cost"

category: behavioral

context: >

Proximity buckets change continuously as time passes.

How frequently must the output CSV be regenerated?

What proximity staleness is acceptable to the EA?

options_evaluated:

- sub_10_second_refresh: rejected (DB hammer; file_seq churn defeats change det
- 60_second_refresh: rejected (IM window for PMI is 10 min; 60s lag = 10% of wi
- 30_second_refresh: selected

selected_option: 30_second_refresh

rules:

- pipeline_scheduler refresh_interval_seconds = 30
- EA StalenessThresholdSeconds = 90 (3 × refresh, tolerates 2 missed cycles)
- Documented tolerance: proximity shown to EA is accurate to ±30 seconds
- For IM events (0-10 min for PMI, 0-15 min default): EA may act on a bucket that is 1 refresh cycle stale. This is accepted and documented.
- If refresh cycle duration_ms > 25000 (25s), emit WARNING log (approaching int
- If refresh cycle duration_ms > 30000 (30s), emit ERROR log (missed cycle)

staleness_contract: >

The EA shall not rely on sub-30-second proximity accuracy.

Any trading logic requiring finer proximity resolution must query

current_time - event_time_utc directly from the CSV's next_event_time_utc field

encoded_in: services/calendar-processor/src/pipeline_scheduler.py

ECONOMIC_CALENDAR_MVP_SPEC.yaml (update staleness_threshold_seconds to

decision_id: DEC-CAL-005

cluster: "Hybrid ID retroactive assembly – generation-O bootstrap"

category: behavioral

context: >

Full hybrid ID requires trade outcome and duration, only known after trade closes

Generation-O trade has no prior outcome. What parameter set does it use?

options_evaluated:

- block_all_trades_until_matrix_populated: rejected (system non-functional at s
- use_fixed_default_regardless_of_cal5: rejected (ignores event-type informatio
- cal5_only_lookup_for_generation_o: selected

selected_option: cal5_only_lookup_for_generation_o

generation_o_lookup_key: "{cal5}:O:DEFAULT"

Example: "10124:O:DEFAULT" for USA Medium impact, generation O

Matrix must have an entry for every valid cal5:O:DEFAULT key

Missing entry → return PARAM_DEFAULT (the universal fallback)

generation_r_lookup_key: "{cal5}:{generation}:{signal_type}:{duration}:{outcome}:"

Example: "10124:R1:ECO_HIGH_USD:FLASH:O4:FSH"

Built only after trade closes and outcome/duration are known

rules:

- Generation O: matrix.lookup(f"{cal5}:O:DEFAULT")
- Generation R1+: matrix.lookup(full_hybrid_id)
- Any lookup miss: return PARAM_DEFAULT; log Decision Registry entry (category=
- PARAM_DEFAULT: conservative parameters (lots=0.01, sl=30pts, tp=20pts, offset

- Decision Registry matrix_miss entries reviewed by operator weekly
rationale: >
System must be operational from day one. Generation-O trades use available information (event region, country, impact level via CAL5). Full hybrid ID lookup activates only once the trade feedback loop has produced outcomes.
encoded_in: services/reentry-matrix-svc/src/matrix_lookup.py

decision_id: DEC-CAL-006
cluster: "file_seq counter has no specified owner"
category: structural
context: >
file_seq must be monotonically increasing across process restarts.
Which service owns it, where is it stored, how does it survive restart?
options_evaluated:
- in_memory_counter: rejected (resets to 1 on restart; EA ignores all signals u
- json_state_file_counter: rejected (race condition if process crashes mid-writ
- sqlite_counter_table: selected
selected_option: sqlite_counter_table
rules:
- Table: file_seq_counters (defined in Phase 0 schema)
- Columns: file_name TEXT PRIMARY KEY, current_seq INTEGER, last_written_at TEX
- On service startup: SELECT current_seq FROM file_seq_counters WHERE file_name
- On each write: UPDATE file_seq_counters SET current_seq = current_seq + 1 in
- Counter is incremented atomically with the file generation inside a single SQ
- On crash recovery: counter reads MAX value from DB; no gap in sequence is pos
ownership:
active_calendar_signals.csv: calendar-processor (owns SQLite)
reentry_decisions.csv: reentry-matrix-svc (owns its own counter in its own SQLi
trade_results.csv: MT4 EA (append-only; no seq counter; identified by ticket nu
encoded_in: services/calendar-processor/src/output_writer.py
services/reentry-matrix-svc/src/decision_writer.py

decision_id: DEC-CAL-007
cluster: "Debounce key breaks at state boundaries"
category: behavioral
context: >
Current debounce key is (symbol, cal8, signal_type, state).
State transitions create new keys, bypassing min-gap at boundaries.
This can produce signal bursts when states transition rapidly.
options_evaluated:
- include_state_in_key: rejected (suppresses legitimate state transition signal

- exclude_state_from_key: selected
- separate_debounce_per_state_window: rejected (over-complex)

selected_option: exclude_state_from_key

debounce_key_definition: "(symbol, cal8, signal_type)"

Excludes: state, proximity, file_seq, timestamp

rules:

- State transitions ALWAYS emit a new signal regardless of min-gap (state change IS the event; suppressing it loses information)
- Min-gap applies only to repeated emissions of identical (symbol, cal8, signal_type) within the SAME STATE
- If (symbol, cal8, signal_type) was emitted < min_gap_minutes ago AND state has NOT changed: suppress emission
- If state HAS changed since last emission: emit regardless of min-gap

implementation: >

```
SELECT MAX(emitted_at) FROM signal_emissions
WHERE debounce_key = ? AND state = ?
If result < NOW() - min_gap_minutes: suppress
Else: emit
```

encoded_in: services/calendar-processor/src/debounce.py

decision_id: DEC-CAL-008

cluster: "priority_weight field undefined"

category: behavioral

context: >

CalendarSignal struct has priority_weight field. No document defines how it is calculated. EA cannot order multiple simultaneous signals.

selected_option: impact_times_proximity_score

formula: "priority_weight = impact_score × proximity_score"

scoring:

impact_score:

- HIGH: 3
- MEDIUM: 2
- ECO_UNCAT: 1 # CHF and other uncat events get minimum score

proximity_score:

- IM: 4 # Most urgent
- SH: 3
- LG: 2
- EX: 1
- CD: 0 # Cooldown — lowest priority

range: "0 (MEDIUM+CD) to 12 (HIGH+IM)"

ea_usage: >

When multiple active signals exist for a symbol, EA uses the signal with the highest priority_weight for next_event metadata display.

Block behavior applies regardless of weight (any active signal blocks).

encoded_in: services/calendar-processor/src/signal_builder.py

decision_id: DEC-CAL-009

cluster: "Matrix population combinatorial explosion"

category: behavioral

context: >

Full hybrid ID space is combinatorial (CAL5 × Generation × SignalType × Duration × Outcome × Proximity). Populating upfront is impractical.
Empty matrix blocks all re-entry trading.

options_evaluated:

- populate_all_combinations_upfront: rejected (thousands of entries; mostly nev
- block_reentry_if_no_entry: rejected (system non-functional; defeats purpose)
- lazy_population_with_default_fallback: selected

selected_option: lazy_population_with_default_fallback

rules:

- matrix.lookup(hybrid_id) returns PARAM_DEFAULT for any unrecognized ID
- Every miss is logged in Decision Registry (category="matrix_miss")
- PARAM_DEFAULT values: lots=0.01, sl_points=30, tp_points=20, offset=0
- Operator reviews matrix_miss log weekly and adds explicit entries for combinations that appear 3+ times (template creation trigger from DES Concept)
- Matrix entries are YAML files in services/reentry-matrix-svc/matrix/ keyed by hybrid_id_pattern (supports glob: "10124:R1:*:*:WIN:SH")
- Explicit entry always overrides default

growth_model: >

Matrix grows organically from observed combinations. After 4 weeks of live trading, the most frequent 20-30 combinations will cover 80%+ of actual traffic

encoded_in: services/reentry-matrix-svc/src/matrix_lookup.py

services/reentry-matrix-svc/matrix/PARAM_DEFAULT.yaml

decision_id: DEC-CAL-010

cluster: "emergency_stop method/attribute name collision (code bug)"

category: behavioral

context: >

EventStateMachine has both a method named emergency_stop() and sets self.emergency_stop = False in __init__. Python resolves self.emergency_stop to the bool after __init__, making the method uncallable.

selected_option: rename_both

renames:

- method: "emergency_stop()" → "trigger_emergency_stop()"
- attribute: "self.emergency_stop" → "self.is_emergency_stopped"

enforcement: >

```
Anti-pattern guard CAL-GUARD-010 runs pylint on state_machine.py
as a pre-flight check on every deployment. Zero tolerance.
acceptance_test: >
python -c "
from state_machine import EventStateMachine
sm = EventStateMachine(None, None)
assert callable(sm.trigger_emergency_stop)
assert isinstance(sm.is_emergency_stopped, bool)
print('PASS')
"
encoded_in: services/calendar-processor/src/state_machine.py
```

2. Anti-Pattern Guards

```
# .execution/anti_patterns.yaml
# EAFIX Economic Calendar – Anti-Pattern Guard Configuration

guards:

CAL-GUARD-001:
  name: stale_output_guard
  cluster: "Proximity output staleness"
  enabled: true
  rule: >
    Output CSV must be regenerated within refresh_interval_seconds.
    File age > 90s triggers stale state.
  implementation: >
    pipeline_scheduler checks (NOW - file mtime) after each cycle.
  thresholds:
    warning_seconds: 45    # approaching interval
    error_seconds: 60     # missed one cycle
    stale_seconds: 90     # EA should treat as missing
  on_warning: log_warning
  on_error: log_error_and_emit_health_metric
  on_stale: >
    Do NOT delete output file (preserve last valid state for EA).
    Log CRITICAL. Emit circuit_breaker candidate event.

CAL-GUARD-002:
  name: write_contention_guard
  cluster: "Two-scheduler SQLite write contention"
  enabled: true
  rule: >
    calendar-downloader process must NEVER open a SQLite connection.
```

```
implementation: >
    Unit test: mock sqlite3.connect in downloader tests; assert never called.
    Integration: run downloader with strace; assert no .db file opens.
on_violation: REJECT_DEPLOYMENT (CI gate, not runtime)
```

CAL-GUARD-003:

```
name: missing_cal8_guard
cluster: "CAL8 identity is the keystone"
enabled: true
rule: >
    No event may pass normalizer output without a valid CAL8 field.
implementation: >
    Normalizer output validation: every normalized_event must have
    cal8 matching regex r'^CAL8-\d{8}-\d{4}-[A-Z]{3}-\d{3}$'
on_violation: REJECT_EVENT (log warning, skip to next row)
```

CAL-GUARD-004:

```
name: generation_o_null_guard
cluster: "Hybrid ID retroactive assembly"
enabled: true
rule: >
    matrix.lookup() must NEVER return None or raise KeyError.
    All misses return PARAM_DEFAULT.
implementation: >
    matrix_lookup.py wraps all lookups in try/except;
    logs Decision Registry entry on miss;
    returns PARAM_DEFAULT object (never None).
on_violation: RETURN_PARAM_DEFAULT_AND_LOG
```

CAL-GUARD-005:

```
name: file_seq_regression_guard
cluster: "file_seq counter has no specified owner"
enabled: true
rule: >
    new file_seq must always be strictly greater than previous file_seq
    for the same output file.
implementation: >
    Before writing: SELECT current_seq FROM file_seq_counters WHERE file_name =
    Assert new_seq = current_seq + 1 within same transaction.
on_violation: ABORT_WRITE, log_critical, emit_circuit_breaker_event
```

CAL-GUARD-006:

```
name: hybrid_id_premature_assembly_guard
cluster: "Hybrid ID retroactive assembly"
enabled: true
rule: >
    Full hybrid ID (including outcome and duration) must not be assembled
```

```
    before the trade's close_time is non-null in trade_results.
implementation: >
    hybrid_id_builder.py asserts trade_result.close_time IS NOT NULL
    before accessing trade_result.outcome and trade_result.duration_minutes.
on_violation: RAISE_ValueError (hard fail; log with trade ticket)
```

CAL-GUARD-007:

```
name: chf_no_cal5_guard
cluster: "CHF exclusion silent gap"
enabled: true
rule: >
    CHF events must have cal5 = "00000". Any non-00000 cal5 on a CHF
    event is an assignment error.
implementation: >
    After CAL5 assignment: assert event.currency != "CHF" or event.cal5 == "000
on_violation: REJECT_EVENT, log_error
```

CAL-GUARD-008:

```
name: proximity_null_guard
cluster: "Proximity output staleness"
enabled: true
rule: >
    proximity_bucket must never be None. Events beyond EX threshold
    receive EX, not None.
implementation: >
    proximity_engine.calculate_proximity() has a final fallback:
    return ProximityBucket.EXTENDED if no bucket matched.
on_violation: RETURN_EXTENDED, log_warning
```

CAL-GUARD-009:

```
name: planning_loop_guard
cluster: "State machine cannot be tested without layers 1 and 2"
enabled: true
rule: >
    Schema definition may not be revised more than 2 times before
    Phase 0 acceptance tests pass. Third revision escalates to human.
implementation: >
    .state/schema_revision_count.json tracks count.
    Phase 0 phase contract enforces this limit.
threshold: 2
on_violation: ESCALATE_TO_HUMAN
```

CAL-GUARD-010:

```
name: name_collision_guard
cluster: "emergency_stop method/attribute name collision"
enabled: true
rule: >
```

```
state_machine.py must pass pylint with zero E-level errors.
Specifically: no method name may shadow an instance attribute.
implementation: >
CI pre-flight: pylint services/calendar-processor/src/state_machine.py
--disable=all --enable=E
accept: "exit code 0"
on_violation: REJECT_DEPLOYMENT
```

3. Single-Writer Authority Registry

```
# .state/single_writer_authority.yaml
# Authoritative registry of which service owns write access to each resource.
# Violations are caught by CAL-GUARD-002 (runtime) and CI gates (deployment).

single_writer_authority:

  calendar_registry.db:
    owner: calendar-processor
    write_mode: exclusive
    other_services_allowed: read_via_query_only
    rationale: "Eliminates SQLite write contention between downloader and process"

  active_calendar_signals.csv:
    owner: calendar-processor
    write_mode: atomic_replace    # temp file → os.replace
    file_seq_counter_location: "calendar_registry.db::file_seq_counters"
    other_services_allowed: read_only
    ea_read_interval_seconds: 5

  reentry_decisions.csv:
    owner: reentry-matrix-svc
    write_mode: atomic_replace
    file_seq_counter_location: "reentry_matrix.db::file_seq_counters"
    other_services_allowed: read_only

  trade_results.csv:
    owner: MT4_EA_CalendarBridge
    write_mode: append_only
    other_services_allowed: read_only
    reader: reentry-engine

  data/raw/:
    owner: calendar-downloader
    write_mode: atomic_replace
```

```

    other_services_allowed: read_only
    reader: calendar-processor (via watcher)

data/download_state.json:
    owner: calendar-downloader
    write_mode: exclusive
    other_services_allowed: none

data/calendar.db:
    owner: calendar-ingestor
    write_mode: exclusive
    note: "Ingestor writes; processor reads via SELECT only"

```

4. Phase 0 Deliverable — Database Schema

This is the blocking deliverable for the entire build. No phase P01–P10 may begin until Phase 0 acceptance tests pass.

```

-- =====
-- EAFIX Economic Calendar — SQLite Schema v1.0.0
-- File: data/calendar_registry.db
-- Owner: calendar-processor (sole writer)
-- =====

PRAGMA journal_mode = WAL;
PRAGMA foreign_keys = ON;

-- Schema version tracking (checked at every service startup)
CREATE TABLE schema_meta (
    key    TEXT PRIMARY KEY,
    value TEXT NOT NULL
);
INSERT INTO schema_meta VALUES ('schema_version', '1.0.0');
INSERT INTO schema_meta VALUES ('created_at', datetime('now', 'utc'));

-- =====
-- Core event registry
-- =====

CREATE TABLE calendar_events (
    cal8            TEXT PRIMARY KEY,      -- CAL8-YYYYMMDD-HHMM-CCC-NNN
    event_date      TEXT NOT NULL,         -- YYYY-MM-DD UTC
    event_time_utc  TEXT NOT NULL,         -- ISO 8601 UTC
    currency        TEXT NOT NULL,         -- 3-letter code
    impact          TEXT NOT NULL          -- HIGH | MEDIUM

```

```

        CHECK (impact IN ('HIGH', 'MEDIUM')),
title      TEXT NOT NULL,
source     TEXT NOT NULL DEFAULT 'forexfactory',
quality_score INTEGER NOT NULL DEFAULT 0
        CHECK (quality_score BETWEEN 0 AND 100),
state      TEXT NOT NULL DEFAULT 'SCHEDULED'
        CHECK (state IN (
            'SCHEDULED', 'ANTICIPATION_8HR', 'ANTICIPATION_1HR',
            'ACTIVE', 'COOLDOWN', 'EXPIRED', 'BLOCKED'
        )),
cal5       TEXT NOT NULL DEFAULT '00000', -- 5-digit RCI or 00000
signal_type TEXT, -- ECO_HIGH_USD | ECO_MED_EUR | ECO_UNCAT
event_type TEXT NOT NULL DEFAULT 'DEFAULT'
        CHECK (event_type IN ('CPI', 'NFP', 'PMI', 'RATE', 'GDP', 'DEFAULT
raw_date_text TEXT,
raw_time_text TEXT,
raw_source_tz TEXT NOT NULL DEFAULT 'America/New_York',
created_at  TEXT NOT NULL DEFAULT (datetime('now', 'utc')),
updated_at  TEXT NOT NULL DEFAULT (datetime('now', 'utc'))
);

```

```

CREATE INDEX idx_events_state      ON calendar_events(state);
CREATE INDEX idx_events_currency  ON calendar_events(currency);
CREATE INDEX idx_events_event_time ON calendar_events(event_time_utc);
CREATE INDEX idx_events_cal5      ON calendar_events(cal5);
CREATE INDEX idx_events_date_currency ON calendar_events(event_date, currency);

```

```

-- =====
-- State transition audit (append-only)
-- =====
CREATE TABLE state_transitions (
    id            INTEGER PRIMARY KEY AUTOINCREMENT,
    cal8          TEXT NOT NULL REFERENCES calendar_events(cal8),
    from_state    TEXT NOT NULL,
    to_state      TEXT NOT NULL,
    reason        TEXT,
    transitioned_at TEXT NOT NULL DEFAULT (datetime('now', 'utc'))
);

```

```

CREATE INDEX idx_transitions_cal8 ON state_transitions(cal8);
CREATE INDEX idx_transitions_ts   ON state_transitions(transitioned_at);

```

```

-- =====
-- Signal emission log (debounce source of truth)
-- =====
CREATE TABLE signal_emissions (
    id            INTEGER PRIMARY KEY AUTOINCREMENT,

```

```

    cal8            TEXT NOT NULL REFERENCES calendar_events(cal8),
    symbol          TEXT NOT NULL,
    signal_type     TEXT NOT NULL,
    proximity       TEXT NOT NULL
                    CHECK (proximity IN ('IM','SH','LG','EX','CD')),
    state          TEXT NOT NULL,
    priority_weight INTEGER NOT NULL CHECK (priority_weight BETWEEN 0 AND 12),
    file_seq        INTEGER NOT NULL,
    debounce_key    TEXT NOT NULL,    -- "{symbol}||{cal8}||{signal_type}"
    emitted_at      TEXT NOT NULL DEFAULT (datetime('now', 'utc'))
);

CREATE INDEX idx_emissions_debounce ON signal_emissions(debounce_key, state);
CREATE INDEX idx_emissions_cal8     ON signal_emissions(cal8);
CREATE INDEX idx_emissions_file_seq ON signal_emissions(file_seq);

-- =====
-- file_seq counters (single source of truth, persists across restarts)
-- =====
CREATE TABLE file_seq_counters (
    file_name      TEXT PRIMARY KEY,
    current_seq    INTEGER NOT NULL DEFAULT 0,
    last_written_at TEXT NOT NULL DEFAULT (datetime('now', 'utc'))
);

INSERT INTO file_seq_counters (file_name, current_seq) VALUES
    ('active_calendar_signals.csv', 0);

-- =====
-- Trade results (written by EA, read by reentry engine)
-- =====
CREATE TABLE trade_results (
    ticket          INTEGER PRIMARY KEY,
    symbol          TEXT NOT NULL,
    cal8            TEXT REFERENCES calendar_events(cal8),
    decision_id     TEXT,
    open_price      REAL NOT NULL,
    close_price     REAL,
    profit          REAL,
    lots           REAL NOT NULL,
    direction       TEXT NOT NULL CHECK (direction IN ('BUY','SELL')),
    parameter_set_id TEXT,
    generation      TEXT CHECK (generation IN ('O','R1','R2')),
    open_time       TEXT NOT NULL,
    close_time      TEXT,          -- NULL until trade closes
    duration_minutes INTEGER,      -- NULL until trade closes
    outcome         TEXT          -- WIN | LOSS | BREAK-EVEN; NULL until clo

```

```

        CHECK (outcome IN ('WIN','LOSS','BREAKEVEN') OR outcome I
created_at      TEXT NOT NULL DEFAULT (datetime('now', 'utc'))
);

CREATE INDEX idx_results_cal8      ON trade_results(cal8);
CREATE INDEX idx_results_outcome ON trade_results(outcome);

-- =====
-- Decision audit registry (DES Concept 9)
-- =====
CREATE TABLE decisions (
    decision_id      TEXT PRIMARY KEY,
    timestamp        TEXT NOT NULL DEFAULT (datetime('now', 'utc')),
    category         TEXT NOT NULL
                    CHECK (category IN (
                        'routing','scheduling','retry',
                        'circuit_breaker','matrix_miss','state_transition'
                    )),
    cal8             TEXT,
    symbol           TEXT,
    context_json     TEXT,
    options_json     TEXT,
    selected_option  TEXT,
    rationale        TEXT,
    outcome          TEXT NOT NULL DEFAULT 'PENDING'
                    CHECK (outcome IN ('PENDING','SUCCESS','FAILURE'))
);

CREATE INDEX idx_decisions_category ON decisions(category);
CREATE INDEX idx_decisions_timestamp ON decisions(timestamp);
CREATE INDEX idx_decisions_outcome ON decisions(outcome);

```

5. Build Phase Contracts (P00–P10)

Each phase contract must pass before the next phase begins. The sequence is the build order mandated by the dependency analysis.

```

# =====
# P00 – Database Schema Initialization
# Blocks: ALL subsequent phases
# =====
phase_id: P00
objective: "Define and initialize the SQLite schema. This is the sole dependency
file_scope:

```

```

creates:
  - data/calendar_registry.db
  - data/schema_v1.sql
modifies: []
must_not_touch:
  - services/ # No service code in Phase 0
dependencies:
  phases: [] # No dependencies - this is the root
pre_flight_checks:
  - "sqlite3 --version exits with code 0"
  - "data/ directory is writable"
acceptance_tests:
  - id: "P00-AT-001"
    description: "Schema applies without error"
    command: "sqlite3 data/calendar_registry.db < data/schema_v1.sql"
    expect_exit_code: 0
  - id: "P00-AT-002"
    description: "All expected tables exist"
    command: >
      sqlite3 data/calendar_registry.db
      "SELECT COUNT(*) FROM sqlite_master WHERE type='table'
      AND name IN ('calendar_events','state_transitions',
        'signal_emissions','file_seq_counters','trade_results','decisions')"
    expect_stdout: "6"
  - id: "P00-AT-003"
    description: "Schema version matches contract"
    command: "sqlite3 data/calendar_registry.db \"SELECT value FROM schema_meta W
    expect_stdout: "1.0.0"
  - id: "P00-AT-004"
    description: "file_seq_counters seeded correctly"
    command: "sqlite3 data/calendar_registry.db \"SELECT current_seq FROM file_se
    expect_stdout: "0"
self_healing:
  auto_fix:
    - condition: "data/ directory does not exist"
      action: "mkdir -p data/"
      reversible: true
  requires_escalation:
    - condition: "Schema applies with errors (column type mismatch etc.)"
      reason: "Schema error requires DEC-CAL revision, not auto-fix"

```

```

# =====
# P01 - CAL8 Assignment + Normalizer
# Blocks: P02 (state machine), P03 (proximity), P04 (CAL5)

```

```

# =====
phase_id: P01
objective: "Implement the event normalizer with CAL8 identity assignment per DEC-
file_scope:
    creates:
        - services/calendar-ingestor/src/normalizer.py
        - tests/calendar-ingestor/test_normalizer.py
    modifies: []
    must_not_touch:
        - services/calendar-processor/
        - services/reentry-matrix-svc/
dependencies:
    phases: [P00]
    schema_version: "1.0.0"
pre_flight_checks:
    - "P00 checkpoint exists and PASS"
    - "sqlite3 data/calendar_registry.db 'SELECT schema_version' returns 1.0.0"
    - "pip show pytz OR pip show zoneinfo exits code 0"
acceptance_tests:
    - id: "P01-AT-001"
      description: "CAL8 format is correct"
      command: "pytest tests/calendar-ingestor/test_normalizer.py::test_cal8_format
      expect_stdout: "1 passed"
    - id: "P01-AT-002"
      description: "DST transition normalized correctly (NORM_001)"
      command: "pytest tests/calendar-ingestor/test_normalizer.py::test_dst_march_t
      expect_stdout: "1 passed"
      note: "8:30am ET on 2025-03-09 must yield 2025-03-09T12:30:00Z"
    - id: "P01-AT-003"
      description: "CHF events receive cal5=00000 (DEC-CAL-002)"
      command: "pytest tests/calendar-ingestor/test_normalizer.py::test_chf_gets_un
      expect_stdout: "1 passed"
    - id: "P01-AT-004"
      description: "CAL8 uniqueness enforced (collision suffix increments)"
      command: "pytest tests/calendar-ingestor/test_normalizer.py::test_cal8_collis
      expect_stdout: "1 passed"
    - id: "P01-AT-005"
      description: "CAL-GUARD-003 fires on event missing CAL8"
      command: "pytest tests/calendar-ingestor/test_normalizer.py::test_missing_cal
      expect_stdout: "1 passed"
self_healing:
    auto_fix:
        - condition: "ModuleNotFoundError: pytz"
          action: "pip install pytz"
          reversible: true
    requires_escalation:
        - condition: "DST test fails (wrong UTC offset)"

```

reason: "Timezone logic error requires developer review; not auto-fixable"

```
# =====
# P02 - Event State Machine
# Blocks: P05 (anticipation signals), P07 (output CSV)
# =====
phase_id: P02
objective: >
    Implement 6-state event lifecycle machine against real SQLite.
    Apply DEC-CAL-010 rename. Implement trigger_emergency_stop().
file_scope:
    creates:
        - services/calendar-processor/src/state_machine.py
        - tests/calendar-processor/test_state_machine.py
    must_not_touch:
        - services/calendar-ingestor/
        - services/reentry-matrix-svc/
dependencies:
    phases: [P00, P01]
pre_flight_checks:
    - "P01 checkpoint exists and PASS"
    - "pylint services/calendar-processor/src/state_machine.py --disable=all --enab
acceptance_tests:
    - id: "P02-AT-001"
      description: "All 6 states reachable in deterministic test mode"
      command: "pytest tests/calendar-processor/test_state_machine.py::test_all_sta
      expect_stdout: "1 passed"
    - id: "P02-AT-002"
      description: "State persists across process restart"
      command: "pytest tests/calendar-processor/test_state_machine.py::test_state_s
      expect_stdout: "1 passed"
    - id: "P02-AT-003"
      description: "trigger_emergency_stop() callable; is_emergency_stopped is bool
      command: "pytest tests/calendar-processor/test_state_machine.py::test_emergen
      expect_stdout: "1 passed"
    - id: "P02-AT-004"
      description: "State transitions written to state_transitions table"
      command: "pytest tests/calendar-processor/test_state_machine.py::test_transit
      expect_stdout: "1 passed"
    - id: "P02-AT-005"
      description: "Invalid transition rejected (no direct SCHEDULED→ACTIVE)"
      command: "pytest tests/calendar-processor/test_state_machine.py::test_invalid
      expect_stdout: "1 passed"
```

```
# =====
# P03 - Proximity Engine + Event Classifier
# Blocks: P05 (anticipation), P07 (output CSV)
# =====
phase_id: P03
objective: >
    Implement event-type classifier and proximity bucket calculator
    per enhancement plan thresholds. Implement DEC-CAL-008 priority_weight.
file_scope:
    creates:
        - services/calendar-processor/src/proximity_engine.py
        - tests/calendar-processor/test_proximity_engine.py
    must_not_touch:
        - services/calendar-ingestor/
dependencies:
    phases: [P00, P01]
acceptance_tests:
    - id: "P03-AT-001"
      description: "NFP classified correctly"
      command: "pytest tests/calendar-processor/test_proximity_engine.py::test_nfp_
        expect_stdout: "1 passed"
    - id: "P03-AT-002"
      description: "NFP IM boundary at 30 min (not 15 min default)"
      command: "pytest tests/calendar-processor/test_proximity_engine.py::test_nfp_
        expect_stdout: "1 passed"
    - id: "P03-AT-003"
      description: "priority_weight = 12 for HIGH+IM, 0 for MEDIUM+CD (DEC-CAL-008)"
      command: "pytest tests/calendar-processor/test_proximity_engine.py::test_prio
        expect_stdout: "1 passed"
    - id: "P03-AT-004"
      description: "CAL-GUARD-008: proximity never returns None"
      command: "pytest tests/calendar-processor/test_proximity_engine.py::test_no_n
        expect_stdout: "1 passed"
```

```
# =====
# P04 - CAL5 / RCI Strategy ID Assignment
# Blocks: P05 (anticipation), P07 (output CSV)
# =====
phase_id: P04
objective: >
    Implement CAL5/RCI generator. Apply DEC-CAL-002 (CHF → 00000).
    Checksum formula: (R + Value(CC) + I) MOD 10.
```

```

file_scope:
  creates:
    - services/calendar-ingestor/src/cal5_generator.py
    - tests/calendar-ingestor/test_cal5_generator.py
  modifies:
    - services/calendar-ingestor/src/normalizer.py # call cal5_generator after C
dependencies:
  phases: [P01]
acceptance_tests:
  - id: "P04-AT-001"
    description: "USA Medium = 10124 (from spec example)"
    command: "pytest tests/calendar-ingestor/test_cal5_generator.py::test_usa_med
    expect_stdout: "1 passed"
  - id: "P04-AT-002"
    description: "CHF always returns 00000 (DEC-CAL-002 + CAL-GUARD-007)"
    command: "pytest tests/calendar-ingestor/test_cal5_generator.py::test_chf_ret
    expect_stdout: "1 passed"
  - id: "P04-AT-003"
    description: "Checksum digit is deterministic (same inputs always same output
    command: "pytest tests/calendar-ingestor/test_cal5_generator.py::test_checksu
    expect_stdout: "1 passed"

```

```

# =====
# P05 - Anticipation Signal Generation
# Depends on: P02 (state machine), P03 (proximity), P04 (CAL5)
# =====
phase_id: P05
objective: >
  Implement anticipation signal emission at ANT_8H and ANT_1H state
  transitions. Wired to state machine callbacks.
file_scope:
  creates:
    - services/calendar-processor/src/anticipation.py
    - tests/calendar-processor/test_anticipation.py
dependencies:
  phases: [P02, P03, P04]
acceptance_tests:
  - id: "P05-AT-001"
    description: "ANT_8H signal emitted when event transitions to ANTICIPATION_8H
    command: "pytest tests/calendar-processor/test_anticipation.py::test_ant_8h_o
    expect_stdout: "1 passed"
  - id: "P05-AT-002"
    description: "ANT_1H signal emitted when event transitions to ANTICIPATION_1H
    command: "pytest tests/calendar-processor/test_anticipation.py::test_ant_1h_o

```

```
    expect_stdout: "1 passed"
- id: "P05-AT-003"
  description: "Signal includes correct cal5, signal_type, priority_weight"
  command: "pytest tests/calendar-processor/test_anticipation.py::test_signal_f
  expect_stdout: "1 passed"
```

```
# =====
# P06 – Debounce / Min-Gap Implementation
# Depends on: P05
# =====
phase_id: P06
objective: >
  Implement debounce suppression per DEC-CAL-007.
  Key: (symbol, cal8, signal_type) WITHOUT state.
  State transitions always emit; min-gap applies within same state only.
file_scope:
  creates:
    - services/calendar-processor/src/debounce.py
    - tests/calendar-processor/test_debounce.py
dependencies:
  phases: [P05]
acceptance_tests:
- id: "P06-AT-001"
  description: "Identical signal within min_gap suppressed"
  command: "pytest tests/calendar-processor/test_debounce.py::test_same_state_s
  expect_stdout: "1 passed"
- id: "P06-AT-002"
  description: "State transition emits despite min_gap not elapsed (DEC-CAL-007
  command: "pytest tests/calendar-processor/test_debounce.py::test_state_transi
  expect_stdout: "1 passed"
```

```
# =====
# P07 – Output CSV Writer with file_seq
# Depends on: P06
# =====
phase_id: P07
objective: >
  Implement atomic output CSV writer. Implement file_seq counter
  per DEC-CAL-006 (SQLite-persisted, restart-safe).
  Refresh cadence: 30 seconds per DEC-CAL-004.
file_scope:
```

```

creates:
  - services/calendar-processor/src/output_writer.py
  - services/calendar-processor/src/pipeline_scheduler.py
  - tests/calendar-processor/test_output_writer.py
dependencies:
  phases: [P06]
acceptance_tests:
  - id: "P07-AT-001"
    description: "file_seq increments monotonically across 3 simulated restarts"
    command: "pytest tests/calendar-processor/test_output_writer.py::test_file_se
    expect_stdout: "1 passed"
  - id: "P07-AT-002"
    description: "Atomic write: partial file never visible to reader"
    command: "pytest tests/calendar-processor/test_output_writer.py::test_atomic_
    expect_stdout: "1 passed"
  - id: "P07-AT-003"
    description: "CAL-GUARD-005: file_seq regression rejected"
    command: "pytest tests/calendar-processor/test_output_writer.py::test_seq_reg
    expect_stdout: "1 passed"
  - id: "P07-AT-004"
    description: "Stale guard warning fires when cycle exceeds 45s"
    command: "pytest tests/calendar-processor/test_output_writer.py::test_stale_g
    expect_stdout: "1 passed"

```

```

# =====
# P08 - CalendarBridge MQL4 (MT4 EA Integration)
# Depends on: P07
# =====
phase_id: P08
objective: >
  Implement CalendarBridge.mqh and EAFX_CalendarIntegrated.mq4.
  Validate against actual P07 output format.
  Implement staleness check per DEC-CAL-004 (90s threshold).
file_scope:
  creates:
    - MT4/Experts/Include/CalendarBridge.mqh
    - MT4/Experts/EAFX_CalendarIntegrated.mq4
    - tests/mt4/test_calendar_bridge_format.py # Python reads file, validates st
dependencies:
  phases: [P07]
acceptance_tests:
  - id: "P08-AT-001"
    description: "Python-written output CSV parsed correctly by bridge format tes
    command: "pytest tests/mt4/test_calendar_bridge_format.py::test_column_order

```

```
    expect_stdout: "1 passed"
- id: "P08-AT-002"
  description: "file_seq monotonicity detected correctly (new data vs no-new-da
  command: "pytest tests/mt4/test_calendar_bridge_format.py::test_file_seq_dete
  expect_stdout: "1 passed"
- id: "P08-AT-003"
  description: "Stale file (age > 90s) triggers fail-safe policy"
  command: "pytest tests/mt4/test_calendar_bridge_format.py::test_staleness_fai
  expect_stdout: "1 passed"
```

```
# =====
# P09 - Trade Result Feedback + Outcome Bucketizer
# Depends on: P08
# =====
phase_id: P09
objective: >
  Implement outcome bucketizer (E1 module).
  WIN/LOSS/BREAKEVEN classification from trade_results.
  Duration categorization: FLASH/QUICK/MEDIUM/LONG/EXTENDED.
  Apply DEC-CAL-006 (CAL-GUARD-006: no hybrid ID before trade closes).
file_scope:
  creates:
    - services/reentry-engine/src/outcome_bucketizer.py # E1
    - services/reentry-engine/src/proximity_evaluator.py # E2
    - tests/reentry-engine/test_outcome_bucketizer.py
dependencies:
  phases: [P08]
acceptance_tests:
- id: "P09-AT-001"
  description: "Trade with profit > pip_threshold classified WIN"
  command: "pytest tests/reentry-engine/test_outcome_bucketizer.py::test_win_cl
  expect_stdout: "1 passed"
- id: "P09-AT-002"
  description: "CAL-GUARD-006: hybrid ID assembly blocked on open trade"
  command: "pytest tests/reentry-engine/test_outcome_bucketizer.py::test_no_hyb
  expect_stdout: "1 passed"
- id: "P09-AT-003"
  description: "Duration bucketing: < 5min = FLASH, 5-15min = QUICK, etc."
  command: "pytest tests/reentry-engine/test_outcome_bucketizer.py::test_durati
  expect_stdout: "1 passed"
```

```

# =====
# P10 - Matrix Lookup + Reentry Engine
# Depends on: P09
# =====
phase_id: P10
objective: >
    Implement matrix lookup per DEC-CAL-009 (lazy, default fallback).
    Implement generation-O lookup per DEC-CAL-005.
    Implement reentry intent builder (E4 module).
file_scope:
    creates:
        - services/reentry-matrix-svc/src/matrix_lookup.py
        - services/reentry-matrix-svc/matrix/PARAM_DEFAULT.yaml
        - services/reentry-engine/src/reentry_intent_builder.py # E4
        - tests/reentry-matrix-svc/test_matrix_lookup.py
dependencies:
    phases: [P09]
acceptance_tests:
    - id: "P10-AT-001"
      description: "Generation-O lookup uses CAL5-only key (DEC-CAL-005)"
      command: "pytest tests/reentry-matrix-svc/test_matrix_lookup.py::test_generat
      expect_stdout: "1 passed"
    - id: "P10-AT-002"
      description: "CAL-GUARD-004: unknown hybrid ID returns PARAM_DEFAULT not None
      command: "pytest tests/reentry-matrix-svc/test_matrix_lookup.py::test_miss_re
      expect_stdout: "1 passed"
    - id: "P10-AT-003"
      description: "Matrix miss logged to Decision Registry (category=matrix_miss)"
      command: "pytest tests/reentry-matrix-svc/test_matrix_lookup.py::test_miss_lo
      expect_stdout: "1 passed"
    - id: "P10-AT-004"
      description: "PARAM_DEFAULT.yaml valid: lots, sl_points, tp_points all presen
      command: "python -c \"import yaml; d=yaml.safe_load(open('services/reentry-ma
      expect_stdout: "PASS"

```

6. Verification Contracts by Layer

```

# Verification contracts summarized by dependency layer.
# Each contract specifies: what is checked, how, and what evidence is written.

verification_contracts:

    layer_1_foundation:

```

layer: "CAL8 + SQLite + Downloader"

checks:

- check_id: "VER-L1-001"
description: "CAL8 uniqueness enforced in DB"
command: >
sqlite3 data/calendar_registry.db
"SELECT cal8, COUNT(*) as c FROM calendar_events GROUP BY cal8 HAVING c
expect_stdout: "" # Empty = no duplicates
evidence_file: ".state/verification/l1_cal8_uniqueness.json"
- check_id: "VER-L1-002"
description: "Downloader has written no SQLite file"
command: "find data/ -name '*.db' -newer data/download_state.json"
expect_stdout: "" # Empty = no db files touched by downloader
evidence_file: ".state/verification/l1_no_db_write.json"
- check_id: "VER-L1-003"
description: "Schema version matches contract"
command: "sqlite3 data/calendar_registry.db \"SELECT value FROM schema_me
expect_stdout: "1.0.0"
evidence_file: ".state/verification/l1_schema_version.json"

layer_2_core:

layer: "State Machine + Proximity Engine + CAL5"

checks:

- check_id: "VER-L2-001"
description: "State transitions persisted for all 6 state changes in inte
command: >
pytest tests/integration/test_state_machine_persistence.py -v
expect_stdout: "6 passed"
evidence_file: ".state/verification/l2_state_persistence.json"
- check_id: "VER-L2-002"
description: "USA Medium CAL5 = 10124"
command: >
python -c "from cal5_generator import generate; assert generate('USD','
expect_stdout: "PASS"
evidence_file: ".state/verification/l2_cal5_example.json"
- check_id: "VER-L2-003"
description: "priority_weight range 0-12 across all combinations"
command: "pytest tests/calendar-processor/test_proximity_engine.py::test_
expect_stdout: "1 passed"

layer_3_signal:

layer: "Anticipation + Debounce + Output CSV"

checks:

- check_id: "VER-L3-001"
description: "file_seq monotonic across 3 restarts"
command: "pytest tests/calendar-processor/test_output_writer.py::test_fil
expect_stdout: "1 passed"

```

    evidence_file: ".state/verification/l3_file_seq.json"
- check_id: "VER-L3-002"
  description: "Debounce: state transition not suppressed despite min-gap"
  command: "pytest tests/calendar-processor/test_debounce.py::test_state_tr
  expect_stdout: "1 passed"
- check_id: "VER-L3-003"
  description: "Output CSV has correct 11-column order"
  command: >
    python -c "
    import csv
    with open('data/active_calendar_signals.csv') as f:
        headers = next(csv.reader(f))
    expected = ['symbol', 'cal8', 'cal5', 'signal_type', 'proximity',
                'event_time_utc', 'state', 'priority_weight',
                'file_seq', 'created_at_utc', 'checksum_sha256']
    assert headers == expected, f'Got {headers}'
    print('PASS')
    "
  expect_stdout: "PASS"

```

layer_4_consumer:

layer: "MT4 EA / CalendarBridge"

checks:

```

- check_id: "VER-L4-001"
  description: "Stale file (>90s) triggers BLOCK behavior in bridge format
  command: "pytest tests/mt4/test_calendar_bridge_format.py::test_staleness
  expect_stdout: "1 passed"
- check_id: "VER-L4-002"
  description: "file_seq unchanged → bridge returns true (no new data) with
  command: "pytest tests/mt4/test_calendar_bridge_format.py::test_no_new_da
  expect_stdout: "1 passed"

```

layer_5_feedback:

layer: "Trade Results + Reentry + Matrix"

checks:

```

- check_id: "VER-L5-001"
  description: "Open trade cannot produce hybrid ID (CAL-GUARD-006)"
  command: "pytest tests/reentry-engine/test_outcome_bucketizer.py::test_no
  expect_stdout: "1 passed"
- check_id: "VER-L5-002"
  description: "Matrix miss returns PARAM_DEFAULT and logs to decisions tab
  command: >
    pytest tests/reentry-matrix-svc/test_matrix_lookup.py::test_miss_return
    tests/reentry-matrix-svc/test_matrix_lookup.py::test_miss_logged -v
  expect_stdout: "2 passed"
- check_id: "VER-L5-003"
  description: "Generation-O uses CAL5-only key format"

```

```
command: "pytest tests/reentry-matrix-svc/test_matrix_lookup.py::test_gen  
expect_stdout: "1 passed"
```

7. What These Rules Cannot Resolve

These three items require human domain knowledge or are fundamental timing constraints that no rule can eliminate.

UNRESOLVED-001: Matrix Parameter Values

Rules can enforce that `PARAM_DEFAULT` exists and that `matrix_miss` is logged. Rules cannot specify what the actual parameter values should be for each trading context (lot sizes, SL/TP distances). These are empirically derived from live trading outcomes.

Resolution path: Operator reviews Decision Registry `matrix_miss` entries weekly. Any combination appearing ≥ 3 times gets an explicit YAML entry in `services/reentry-matrix-svc/matrix/`. This is a data problem, not an architecture problem.

UNRESOLVED-002: The 30-Second Proximity Window

DEC-CAL-004 sets a 30-second refresh cadence. For events in the PMI IM window (0-10 minutes), a trade that executes in the 30-second gap between proximity bucket transitions (e.g., SH→IM at exactly the boundary) may be assigned the wrong parameter set for that re-entry. No rule prevents this — it is a fundamental timing uncertainty.

Documented tolerance: ± 30 seconds of proximity inaccuracy is accepted. If this is unacceptable for a specific event type, the pipeline refresh interval must be reduced and DB load re-evaluated. This is a performance trade-off, not an architecture problem.

UNRESOLVED-003: Reentry Engine Internal Logic (E1/E2/E4)

The module names `E1_OUTCOME_BUCKETIZER`, `E2_PROXIMITY_EVALUATOR`, `E4_REENTRY_INTENT_BUILDER` appear in the file registry. Their internal classification algorithms (what pip threshold constitutes WIN, how duration boundaries are set, how reentry intent is constructed) are not specified in any document in the corpus.

Phase contracts P09 and P10 enforce their input/output contracts (what goes in, what must come out). The internal logic requires a separate specification document before

implementation can begin.

Resolution path: Create `EAFIX_REENTRY_ENGINE_SPEC.yaml` covering E1/E2/E4 algorithms before P09 build begins. Use the same DES phase contract format as this document.

Coverage Summary

Complexity Cluster	Rule(s) That Fix It	Status
CAL8 identity is keystone	DEC-CAL-001, CAL-GUARD-003, P01	RESOLVED
Two-scheduler write contention	DEC-CAL-003, CAL-GUARD-002, Single-Writer Registry	RESOLVED
State machine layer dependency	P00→P01→P02 phase contract chain	RESOLVED
Proximity staleness vs. refresh	DEC-CAL-004, CAL-GUARD-001	RESOLVED (with accepted tolerance)
Hybrid ID retroactive assembly	DEC-CAL-005, CAL-GUARD-006, P10	RESOLVED
file_seq counter ownership	DEC-CAL-006, CAL-GUARD-005, P07, Schema	RESOLVED
Debounce key at state boundaries	DEC-CAL-007, P06	RESOLVED
CHF silent coverage gap	DEC-CAL-002, CAL-GUARD-007, P01/P04	RESOLVED
priority_weight undefined	DEC-CAL-008, P03	RESOLVED
Matrix combinatorial explosion	DEC-CAL-009, CAL-GUARD-004, P10	RESOLVED
emergency_stop name collision	DEC-CAL-010, CAL-GUARD-010, P02	RESOLVED
Database schema undefined	Phase 0 Schema, P00 contract	RESOLVED

Matrix parameter values	—	UNRESOLVED (data problem)
30-second timing gap	DEC-CAL-004 (documented tolerance)	ACCEPTED
E1/E2/E4 internal algorithms	—	UNRESOLVED (needs spec doc)

Result: 12 of 15 problems fully resolved by rules. 2 require domain knowledge (matrix values, E1/E2/E4 logic). 1 is a documented physical constraint accepted by design.